

Unit-4

Test Planning:

Introduction to Test Planning

Test Planning is the first and most important phase of the software testing life cycle (STLC). It defines the **overall strategy, scope, resources, schedule, and deliverables** required to perform testing activities effectively.

A well-prepared test plan acts as a **master document** that guides the entire testing process and ensures that testing is carried out in a **systematic and controlled manner**.

Definition of Test Plan

A **Test Plan** is a **formal document** that describes:

- The **testing strategy**
- The **scope and objectives of testing**
- Required **resources and schedule**
- Planned **test deliverables**

It clearly states **what will be tested, how it will be tested, who will test it, and when testing will be completed**.

Preparing a Test Plan

Introduction

Preparing a Test Plan is the most important activity in the **Test Planning phase** of the Software Testing Life Cycle (STLC).

A test plan acts as a **blueprint** for all testing activities and ensures that testing is conducted in a **systematic, controlled, and effective manner**.

The test plan is prepared **before test execution** and is usually created by the **QA Lead or Test Manager**.

Definition

A **Test Plan** is a **formal document** that describes the **testing objectives, scope, approach, resources, schedule, risks, and deliverables** for a software project.

It answers the following questions:

- What will be tested?
- How will it be tested?
- Who will perform testing?
- When will testing be completed?

Objectives of Preparing a Test Plan

- To define the **scope and objectives** of testing
- To identify **testing strategy and techniques**
- To estimate **time, effort, and cost**
- To allocate **resources and responsibilities**
- To reduce **testing risks**
- To ensure effective **stakeholder communication**

Steps Involved in Preparing a Test Plan

1. Understanding Requirements

Explanation:

The first step is to thoroughly study:

- Software Requirement Specification (SRS)
- Functional and non-functional requirements
- Business requirements

This helps in identifying:

- Testable features
- Testing scope
- Risks and constraints

Importance:

- Ensures correct requirement coverage
- Avoids missing critical test scenarios

2. Defining Test Scope

Explanation:

In this step, the scope of testing is defined by identifying:

- Features to be tested
- Features not to be tested
- Assumptions and limitations

Example:

Included: Login, Registration

Excluded: Third-party payment gateway

3. Identifying Test Objectives

Explanation:

Test objectives define **what the testing process aims to achieve**.

Examples:

- Verify functional correctness
- Validate performance and security
- Ensure compliance with requirements

4. Selecting Test Approach

Explanation:

The test approach defines **how testing will be carried out**.

It includes:

- Types of testing (Unit, Integration, System, Acceptance)
- Test techniques (Black box, White box)
- Manual or automated testing
- Test tools to be used

5. Defining Entry and Exit Criteria

Entry Criteria:

Conditions to start testing:

- Requirements approved
- Test cases prepared
- Test environment ready

Exit Criteria:

Conditions to stop testing:

- All test cases executed
- Critical defects fixed
- Test coverage achieved

6. Resource and Schedule Planning

Explanation:

This step involves identifying:

- Test team members and roles
- Test environment requirements
- Tools and infrastructure
- Testing timeline and milestones

Example:

- 3 Testers

- 1 Automation Engineer
- Testing duration: 4 weeks

7. Identifying Test Deliverables

Explanation:

Test deliverables are documents produced during testing.

Examples:

- Test Plan
- Test Cases
- Defect Reports
- Test Summary Report

8. Risk Identification and Mitigation

Explanation:

Potential risks are identified and mitigation strategies are planned.

Examples:

Risk	Mitigation
Requirement changes	Flexible test plan
Resource unavailability	Backup resources

9. Review and Approval

Explanation:

The completed test plan is:

- Reviewed by stakeholders
- Approved by project manager and client

This ensures alignment with project goals.

Diagram: Preparing a Test Plan

Requirements Analysis



Define Test Scope



Set Test Objectives



Select Test Approach



Plan Resources & Schedule



Identify Risks



Review & Approval

Importance of Preparing a Test Plan

- Provides clarity and direction
- Improves test coverage
- Reduces project risks
- Helps in cost and effort estimation
- Acts as a reference document

Preparing a test plan is a **critical activity** that lays the foundation for successful software testing.

A well-prepared test plan ensures that testing is **organized, efficient, transparent, and aligned with project objectives**, leading to the delivery of **high-quality software products**.

test Approach,

Introduction

A **Test Approach** is an important part of the **Test Plan** that defines **how testing will be carried out** for a software project.

It provides a high-level description of the **testing strategy, methods, techniques, tools, and resources** to be used during the testing process.

The test approach ensures that testing activities are **consistent, systematic, and aligned with project objectives**.

Definition

A **Test Approach** is a section of the test plan that describes the **overall testing strategy**, including:

- Testing levels
- Testing types
- Testing techniques
- Tools and environment
- Roles and responsibilities

It explains **how the testing objectives will be achieved**.

Objectives of Test Approach

- To define a **clear testing strategy**
- To select appropriate **testing techniques**
- To ensure effective **test coverage**
- To standardize testing activities
- To reduce testing risks

Components of Test Approach

1. Testing Levels

Testing levels define **at which stage testing is performed.**

a) Unit Testing

- Performed on individual modules
- Usually done by developers
- Focuses on internal logic

b) Integration Testing

- Tests interaction between modules
- Identifies interface defects

c) System Testing

- Tests the complete system
- Performed by test engineers

d) Acceptance Testing

- Validates system against user requirements
- Performed by customers/end users

2. Types of Testing

Testing types describe **what aspect of the software is tested.**

Functional Testing

- Verifies functionality against requirements
- Example: Login, registration

Non-Functional Testing

- Tests performance, security, usability
- Example: Load testing, stress testing

3. Testing Techniques

Testing techniques define **how test cases are designed**.

a) Black Box Testing

- No knowledge of internal code
- Focuses on input and output

b) White Box Testing

- Requires knowledge of code structure
- Focuses on logic and paths

c) Gray Box Testing

- Combination of black and white box testing

4. Test Execution Strategy

Defines **how test cases will be executed**.

Includes:

- Manual testing
- Automated testing
- Regression testing
- Retesting after defect fixes

5. Tools and Test Environment

Testing Tools

- Manual testing tools
- Automation tools (Selenium, JUnit)

Test Environment

- Hardware
- Software
- Operating system
- Browsers and databases

6. Roles and Responsibilities

Defines **who performs which testing activity**.

Role	Responsibility
QA Lead	Test planning and monitoring
Tester	Test case execution
Automation Engineer	Automation scripting

Diagram: Test Approach

Test Approach

Testing Levels	
Testing Types	
Testing Techniques	
Execution Strategy	
Tools & Environment	
Roles & Responsibilities	

Importance of Test Approach

- Ensures structured testing
- Improves quality and coverage
- Reduces ambiguity
- Aligns testing with development process

Example of Test Approach (Short)

- Testing Levels: System and Acceptance
- Technique: Black box testing
- Execution: Manual testing for UI, automation for regression
- Tools: Selenium
- Environment: Windows, Chrome browser

Advantages of a Well-Defined Test Approach

- Better test planning
- Efficient resource usage
- Faster defect detection
- Improved stakeholder confidence

Conclusion

The test approach is a **core component of the test plan** that defines the **overall strategy for testing**.

A well-defined test approach ensures that testing is **systematic, efficient, and aligned with project goals**, leading to the delivery of **high-quality software**.

criteria for Testing:

Introduction

Criteria for Testing define the **conditions that must be satisfied before testing can begin and before testing can be completed.**

These criteria help in **controlling, monitoring, and evaluating** the testing process and ensure that testing activities are conducted in a **systematic and measurable manner.**

Testing criteria are an important part of the **Test Plan.**

Definition

Testing Criteria are a set of **predefined conditions** that determine:

- When testing should **start**
- When testing should **stop**

They ensure that testing is neither started prematurely nor stopped without achieving quality goals.

Types of Testing Criteria

There are **two main types** of testing criteria:

1. **Entry Criteria**
2. **Exit Criteria**

1. Entry Criteria

Definition:

Entry Criteria are the conditions that must be met **before test execution can begin.**

They ensure that the testing environment is ready and the input artifacts are complete and approved.

Common Entry Criteria

- Software requirements are **reviewed and approved**
- Test plan and test cases are **prepared and reviewed**
- Test environment is **set up and stable**
- Test data is **available**
- Build is **successfully installed**
- Required tools are **configured**

Importance of Entry Criteria

- Prevents waste of testing effort
- Reduces rework

- Ensures readiness for testing
- Improves testing efficiency

Example:

Testing should not start if:

- Requirements are incomplete
- Build is unstable
- Test environment is not ready

2. Exit Criteria

Definition:

Exit Criteria are the conditions that must be satisfied **to conclude testing activities**.

They help decide whether the software is ready for release or further testing is required.

Common Exit Criteria

- All planned test cases are **executed**
- Critical and high-severity defects are **fixed and verified**
- Test coverage goals are **achieved**
- Regression testing is **completed**
- Test summary report is **prepared**
- Stakeholder approval is obtained

Importance of Exit Criteria

- Ensures quality standards are met
- Avoids premature release
- Provides objective release decision
- Confirms completion of testing

Example:

Testing can be stopped when:

- 95% test cases pass
- No critical defects remain open

Diagram: Criteria for Testing

Entry Criteria



Test Execution



Exit Criteria

Entry vs Exit Criteria (Comparison)

Aspect	Entry Criteria	Exit Criteria
Purpose	Start testing	Stop testing
Focus	Readiness	Completion
Defined in	Test Plan	Test Plan
Examples	Test cases ready	All tests executed

Benefits of Defining Testing Criteria

- Clear testing boundaries
- Improved test control
- Reduced risk of defects
- Better project planning
- Objective decision making

Conclusion

Criteria for testing play a **crucial role** in managing the testing process.

By clearly defining **entry and exit criteria**, organizations can ensure that testing starts at the right time and ends only when **quality objectives are achieved**, leading to reliable and high-quality software products.

Test Deliverables,

Introduction

Test Deliverables are the **documents, reports, and outputs** produced during various phases of the **Software Testing Life Cycle (STLC)**.

They provide **evidence of testing activities**, help in tracking progress, and support **decision-making** regarding software quality and release readiness.

Test deliverables are defined during the **test planning phase** and are submitted to stakeholders at different stages of testing.

Definition

Test Deliverables are the **artifacts created as a result of testing activities** that describe:

- What testing was planned
- How testing was executed
- What defects were found
- The final status of the software

Objectives of Test Deliverables

- To document testing activities
- To ensure traceability between requirements and test cases
- To communicate testing progress and results
- To support audit and compliance
- To assist in release decisions

Types of Test Deliverables

Test deliverables are categorized based on **testing phases**.

1. Test Planning Phase Deliverables

a) Test Plan

- Describes testing scope, approach, resources, and schedule
- Acts as a master document for testing

b) Test Strategy

- High-level approach to testing
- Defines testing types and standards

2. Test Design Phase Deliverables

a) Test Scenarios

- High-level test conditions
- Derived from requirements

b) Test Cases

- Step-by-step procedures to test functionalities
- Include inputs, expected results, and execution status

c) Test Data

- Input data used for executing test cases

d) Requirements Traceability Matrix (RTM)

- Maps requirements to test cases
- Ensures full test coverage

3. Test Execution Phase Deliverables

a) Test Execution Report

- Status of executed test cases
- Pass/fail statistics

b) Defect / Bug Report

- Details of identified defects
- Includes severity, priority, and status

4. Test Closure Phase Deliverables

a) Test Summary Report

- Overall summary of testing activities
- Defect metrics and test coverage

b) Test Closure Report

- Confirms completion of testing
- Includes lessons learned

Diagram: Test Deliverables Across STLC

Test Planning → Test Plan
Test Design → Test Cases, RTM
Test Execution → Defect Reports
Test Closure → Test Summary Report

Importance of Test Deliverables

- Provide transparency and accountability
- Help track testing progress
- Improve communication with stakeholders
- Serve as reference for future projects
- Ensure quality compliance

Example of Test Deliverables

For a Login Module:

- Test Plan: Defines login testing strategy
- Test Case: Verify login with valid credentials
- Defect Report: Invalid error message displayed
- Test Summary Report: Login module tested successfully

Advantages of Test Deliverables

- Structured documentation
- Measurable testing outcomes
- Improved software quality
- Supports audits and reviews

Test deliverables are **essential artifacts** that document the **planning, execution, and completion** of testing activities.

They provide **visibility into software quality**, ensure traceability, and help stakeholders make informed release decisions.

Test Case Specification.

Introduction

A **Test Case Specification** defines the **detailed conditions and steps** under which a software application is tested to verify that it meets specified requirements.

It is one of the most important documents prepared during the **test design phase** of the Software Testing Life Cycle (STLC).

Test case specification ensures that testing is **systematic, repeatable, and measurable**.

Definition

A **Test Case** is a **documented set of inputs, execution conditions, and expected results** developed to verify a particular requirement or functionality of a software system.

Test Case Specification refers to the **complete description and documentation of test cases** in a standard format.

Objectives of Test Case Specification

- To verify functional and non-functional requirements
- To ensure maximum test coverage
- To detect defects effectively
- To enable repeatable and consistent testing
- To support regression and automation testing

When Test Cases are Prepared

- After requirements analysis
- During the **test design phase**
- Before test execution begins

Components of Test Case Specification

A standard test case specification contains the following fields:

1. Test Case ID

- A **unique identifier** for each test case
- Helps in tracking and reporting

Example: TC_Login_01

2. Test Case Description

- Brief description of what is being tested
- Written in clear and simple language

Example: Verify login with valid credentials

3. Preconditions

- Conditions that must be true before executing the test case
- Example: User must be registered

4. Test Steps

- Step-by-step instructions to execute the test
- Written in sequential order

5. Test Data / Input

- Input values used during execution

- Can be valid or invalid data

6. Expected Result

- Expected system behavior after executing test steps
- Used to decide pass or fail

7. Actual Result

- Actual behavior observed during test execution

8. Test Status

- Indicates result of execution
- Values: **Pass** / **Fail** / **Blocked** / **Not Executed**

9. Remarks / Comments

- Additional observations or notes

Diagram: Test Case Specification Format

+-----+	
Test Case ID	
Test Case Description	
Preconditions	
Test Steps	
Test Data	
Expected Result	
Actual Result	
Status (Pass/Fail)	
Remarks	
+-----+	

Example of Test Case Specification

Test Case: Login Functionality

Field	Description
Test Case ID	TC_Login_01
Description	Verify login with valid credentials
Preconditions	User must be registered
Test Steps	1. Enter username 2. Enter password 3. Click Login
Test Data	Username: user1 Password: pass123
Expected Result	User should be logged in successfully
Actual Result	User logged in
Status	Pass

Characteristics of Good Test Case Specification

- Clear and unambiguous
- Easy to understand
- Reusable
- Traceable to requirements
- Maintainable

Importance of Test Case Specification

- Ensures complete requirement coverage
- Improves defect detection
- Supports regression testing
- Helps in automation
- Acts as test documentation

Test Case vs Test Scenario

Test Case	Test Scenario
Detailed steps	High-level condition

Test Case	Test Scenario
Focus on execution	Focus on coverage
Time-consuming	Less time required

Advantages of Test Case Specification

- Improves testing quality
- Enables systematic testing
- Helps new testers understand application
- Provides evidence of testing

Test Case Specification is a **critical document** in software testing that defines **how testing is performed** at a detailed level.

Well-written test cases ensure **effective defect detection, high test coverage, and reliable software quality**.

Test Reporting- Executing Test Cases, Preparing Test Summary Report.

Introduction

Test Reporting is an important activity in the **Software Testing Life Cycle (STLC)** that provides information about the **status, progress, and results of testing activities**.

It helps stakeholders understand the **quality of the software** and supports decision-making regarding release readiness.

Test reporting mainly includes:

1. **Executing Test Cases**
2. **Preparing Test Summary Report**

1. Executing Test Cases

Definition

Executing Test Cases is the process of **running test cases on the software application** and comparing the **actual results** with the **expected results** to identify defects.

This activity is performed during the **test execution phase**.

Objectives of Test Case Execution

- To validate software functionality
- To identify defects
- To verify requirement compliance
- To collect test execution results

Steps Involved in Executing Test Cases

1. Test Environment Setup

- Configure hardware, software, tools, and network
- Install test build

2. Test Case Execution

- Execute test cases as per test plan
- Follow defined test steps

3. Result Comparison

- Compare actual results with expected results

4. Status Update

- Mark test case as Pass / Fail / Blocked

5. Defect Logging

- Log defects for failed test cases
- Assign severity and priority

6. Retesting and Regression Testing

- Retest fixed defects
- Perform regression testing to ensure no new defects

Diagram: Test Case Execution Process

Prepare Test Environment

↓

Execute Test Cases

↓

Compare Results

↓

Pass

↓

Fail

↓

Update Status

↓

Log Defect

Example of Test Execution

Test Case: Login with valid credentials
Expected Result: Successful login
Actual Result: Error message displayed
Status: Fail
Action: Defect logged

Importance of Test Case Execution

- Confirms software correctness
- Detects functional and non-functional defects
- Provides input for test reporting
- Improves software reliability

2. Preparing Test Summary Report

Definition

A **Test Summary Report** is a **formal document** that provides a **high-level summary of all testing activities** performed during a testing phase or project.

It is prepared at the **end of testing** and supports the **final release decision**.

Objectives of Test Summary Report

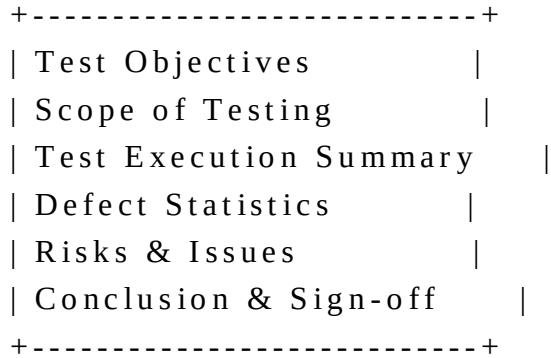
- To summarize test execution results
- To report defect status
- To evaluate test coverage
- To provide quality assessment
- To support management decision-making

Contents of Test Summary Report

1. Test objectives
2. Scope of testing
3. Test environment details
4. Test execution summary
5. Defect statistics
6. Test coverage analysis
7. Risks and issues
8. Limitations and assumptions

- 9. Conclusion and recommendations
- 10. Approval and sign-off

Diagram: Test Summary Report Structure



Example of Test Summary Data

Item	Details
Total Test Cases	200
Executed	190
Passed	175
Failed	15
Critical Defects	0
Status	Ready for release

Importance of Test Summary Report

- Provides overall quality assessment
- Helps management decide release readiness
- Acts as official test completion document
- Useful for audits and future reference

Executing Test Cases vs Test Summary Report

Aspect	Executing Test Cases	Test Summary Report
Phase	Test Execution	Test Closure
Purpose	Find defects	Summarize testing
Output	Pass/Fail results	Final quality status

Conclusion

Test reporting plays a **critical role** in software testing by ensuring transparency and accountability.

Executing test cases validates software behavior, while the test summary report provides a **comprehensive view of software quality**, enabling informed release decisions.

Manual Testing vs Automated Testing Tools.

Introduction

Software testing can be performed either **manually by human testers** or **automatically using software tools**.

Both **Manual Testing** and **Automated Testing** play an important role in ensuring the **quality, reliability, and correctness** of software applications.

Choosing the appropriate testing method depends on **project requirements, budget, timeline, and complexity**.

Manual Testing

Definition

Manual Testing is a type of software testing in which **test cases are executed manually by testers** without using any automation tools.

The tester plays the role of an end user and verifies the application's behavior by interacting with it.

Characteristics of Manual Testing

- No automation tools required
- Human observation and judgment involved
- Flexible and exploratory
- Suitable for early testing stages

Types of Manual Testing

- Unit Testing
- System Testing
- User Acceptance Testing (UAT)
- Exploratory Testing
- Ad-hoc Testing

Advantages of Manual Testing

- Easy to start with minimal cost
- Better for usability and UI testing
- Suitable for exploratory testing
- Detects visual and user-experience issues

Limitations of Manual Testing

- Time-consuming
- Error-prone
- Not suitable for repetitive testing
- Difficult to scale

Example

Manually testing a login page by entering different usernames and passwords and observing the results.

Automated Testing

Definition

Automated Testing is a type of software testing in which **test cases are executed automatically using testing tools or scripts**. It reduces human intervention and increases **speed, accuracy, and repeatability**.

Characteristics of Automated Testing

- Uses tools and scripts
- Faster execution
- High accuracy
- Suitable for repetitive tasks

Automated Testing Tools

Some commonly used automated testing tools are:

- **Selenium** – Web application testing
- **JUnit** – Unit testing (Java)
- **TestNG** – Advanced test framework
- **QTP/UFT** – Functional testing
- **LoadRunner** – Performance testing

Advantages of Automated Testing

- Saves time in regression testing
- High accuracy and reliability
- Reusable test scripts
- Supports Continuous Integration (CI)

Limitations of Automated Testing

- High initial cost
- Requires skilled resources
- Not suitable for usability testing
- Maintenance of scripts needed

Example

Using Selenium scripts to automatically test login functionality for multiple users.

Diagram: Manual vs Automated Testing

Manual Testing	Automated Testing
-----	-----
Human Execution	Tool Execution
Slow	Fast
Exploratory	Regression
Low Initial Cost	High Initial Cost

Comparison: Manual Testing vs Automated Testing Tools

Aspect	Manual Testing	Automated Testing
Execution	Human testers	Tools and scripts
Speed	Slow	Fast
Accuracy	Error-prone	High accuracy
Cost	Low initial cost	High initial cost
Best suited for	Exploratory, UI testing	Regression, load testing
Reusability	No	Yes
Tools	Not required	Selenium, QTP, TestNG
Scalability	Low	High

When to Use Manual Testing

- New application testing
- UI/UX validation
- Exploratory testing
- Short-term projects

When to Use Automated Testing

- Regression testing
- Large and complex systems
- Performance and load testing
- Agile and DevOps environments

Conclusion

Both **Manual Testing** and **Automated Testing Tools** are essential for effective software testing.

Manual testing is suitable for **human-centric evaluation**, while automated testing provides **speed, accuracy, and efficiency** for repetitive tasks.

A **balanced combination of both approaches** ensures high-quality software delivery.

Metrics and Measurement: Types of Metrics, Product

1. Introduction to Metrics and Measurement

In software testing, **metrics and measurement** are used to **quantify quality, productivity, and effectiveness** of the software product and the testing process.

They help in **monitoring progress, improving decision-making, and ensuring quality goals are met**.

2. Definition

Measurement

Measurement is the process of **collecting quantitative data** about software attributes such as size, defects, effort, and performance.

Metric

A **metric** is a **quantitative measure** used to evaluate:

- Software quality
- Testing effectiveness

- Process efficiency

3. Objectives of Metrics and Measurement

- To measure software quality objectively
- To track testing progress
- To identify problem areas
- To improve process efficiency
- To support management decisions

4. Types of Metrics

Software metrics are broadly classified into the following types:

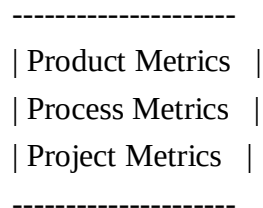
1. Product Metrics

2. Process Metrics

3. Project Metrics

Diagram: Types of Metrics

Software Metrics



5. Product Metrics

Definition

Product Metrics measure the **quality and characteristics of the software product** itself. They focus on **what is being built**, rather than how it is built.

Product metrics help evaluate:

- Reliability
- Maintainability
- Usability
- Performance

Objectives of Product Metrics

- To assess software quality
- To identify defect-prone areas

- To improve product reliability
- To ensure customer satisfaction

Common Product Metrics

1. Defect Density

Definition:

Defect density measures the **number of defects per unit size** of software.

Formula:

Defect Density = Number of Defects

Size of Software

(Size may be measured in LOC or Function Points)

Importance:

- Indicates quality level
- Higher value = lower quality

2. Defect Severity Index

Definition:

Measures the **impact of defects** on the system.

Example:

- Critical defects: System crash
- Minor defects: UI alignment issue

3. Code Coverage

Definition:

Measures the **percentage of code executed** during testing.

Formula:

Code Coverage (%) =

Executed Code / Total Code × 100

Importance:

- Ensures thorough testing
- Identifies untested code

4. Reliability Metric

Definition:

Measures the **ability of software to perform without failure** for a given time.

Example:

Mean Time Between Failures (MTBF)

5. Maintainability Metric

Definition:

Measures how easily the software can be **modified or corrected**.

Indicators:

- Code complexity
- Documentation quality

Diagram: Product Metrics Focus

Product Metrics

| Defects |
| Code Quality |
| Reliability |
Maintainability

Advantages of Product Metrics

- Improves software quality
- Helps in early defect detection
- Supports quality assurance
- Provides measurable results

Limitations of Product Metrics

- May not capture user experience fully
- Incorrect data leads to wrong conclusions
- Needs proper interpretation

Example of Product Metrics Usage

For an e-commerce application:

- Defect Density = 0.5 defects/KLOC
- Code Coverage = 92%
- Result: High-quality product

Difference Between Measurement and Metric

Measurement	Metric
Data collection	Data evaluation
Raw values	Processed values
Example: number of defects	Example: defect density

Conclusion

Metrics and measurement play a **vital role in software testing** by providing **objective insight into product quality**.

Among different types, **product metrics** are essential for assessing the **reliability, correctness, and maintainability** of software, helping organizations deliver **high-quality products**.

Metrics and Process Metrics, Object oriented metrics in testing.

1. Introduction

In software testing, **metrics and measurement** are used to **quantify the effectiveness, efficiency, and quality** of testing activities and software products.

Among different types of metrics, **Process Metrics** and **Object-Oriented (OO) Metrics** play a vital role in improving **testing processes and software quality**.

2. Process Metrics

Definition

Process Metrics measure the **effectiveness and efficiency of the software testing process**.

They focus on **how well testing activities are performed**, rather than on the software product itself.

Process metrics help in:

- Monitoring testing progress
- Improving testing processes
- Reducing defects and cost

Objectives of Process Metrics

- To evaluate testing efficiency
- To monitor test execution progress
- To improve defect detection capability
- To optimize testing effort and cost

- To support continuous process improvement

Common Process Metrics

1. Test Case Execution Rate

Definition:

Measures the number of test cases executed in a given time period.

Formula:

Test Case Execution Rate =

Number of Test Cases Executed / Time

Importance:

- Tracks testing progress
- Helps in schedule management

2. Defect Removal Efficiency (DRE)

Definition:

Measures how effectively defects are detected and removed **before software release**.

Formula:

DRE =

Defects Removed Before Release

----- × 100

Total Defects

Importance:

- Indicates testing effectiveness
- Higher DRE = better quality

3. Test Effort Variance

Definition:

Measures the difference between **planned effort and actual effort**.

Formula:

Effort Variance =

Actual Effort – Planned Effort

Importance:

- Helps in effort estimation
- Identifies process inefficiencies

4. Defect Leakage

Definition:

Defects found in later phases that were **missed in earlier phases**.

Importance:

- Indicates weakness in testing process

5. Requirement Coverage

Definition:

Measures the percentage of requirements covered by test cases.

Formula:

Requirement Coverage (%) =

Tested Requirements / Total Requirements × 100

Diagram: Process Metrics

Process Metrics

| Test Progress |

| Defect Removal |

| Effort Variance |

| Requirement Coverage |

Advantages of Process Metrics

- Improves testing efficiency
- Helps identify process gaps
- Supports continuous improvement
- Reduces development cost

Limitations of Process Metrics

- May not reflect product quality directly
- Requires accurate data collection
- Can be misinterpreted

3. Object-Oriented Metrics in Testing

Definition

Object-Oriented (OO) Metrics measure the **quality, complexity, and maintainability** of object-oriented software systems.

They help testers identify **complex, fault-prone classes** and prioritize testing efforts.

Objectives of OO Metrics

- To measure design complexity
- To identify error-prone classes
- To improve test planning and coverage
- To enhance maintainability

Common Object-Oriented Metrics

1. WMC (Weighted Methods per Class)

Definition:

Measures the **complexity of a class** by counting the number of methods.

Interpretation:

- High WMC → High complexity → More testing required

2. DIT (Depth of Inheritance Tree)

Definition:

Measures the **depth of inheritance hierarchy** of a class.

Interpretation:

- Greater DIT → Increased complexity
- Inherited defects possible

3. NOC (Number of Children)

Definition:

Measures the number of **subclasses** derived from a class.

Interpretation:

- High NOC → Greater reuse but more testing needed

4. CBO (Coupling Between Objects)

Definition:

Measures the **degree of interdependence between classes**.

Interpretation:

- High CBO → Higher risk of defects
- More integration testing required

5. LCOM (Lack of Cohesion of Methods)

Definition:

Measures how well the methods of a class are related.

Interpretation:

- High LCOM → Poor cohesion → Low quality

Diagram: Object-Oriented Metrics

Object-Oriented Metrics

| WMC – Class Complexity |
| DIT – Inheritance Depth|
| NOC – Subclasses |
| CBO – Coupling |
LCOM – Cohesion

Importance of OO Metrics in Testing

- Identifies complex classes
- Helps prioritize test cases
- Improves maintainability
- Reduces defect density

Comparison: Process Metrics vs OO Metrics

Aspect	Process Metrics	OO Metrics
Focus	Testing process	Software design
Measures	Efficiency	Complexity
Used by	Test managers	Testers & developers
Example	DRE	WMC

Conclusion

Process Metrics help evaluate and improve the **effectiveness of testing activities**, while **Object-Oriented Metrics** help assess **design quality and complexity** of software systems. Together, they enable **better test planning, higher quality software, and continuous improvement** in software testing.

Agile Testing: Agile Testing,

1. Introduction

Agile Testing is a software testing practice that follows the **Agile development methodology**, where testing is **continuous, iterative, and integrated** throughout the software development life cycle (SDLC).

Unlike traditional testing, Agile Testing starts **from the beginning of the project** and continues with each iteration (sprint).

Agile Testing ensures **early defect detection, faster feedback, and high-quality software delivery**.

2. Definition

Agile Testing is a testing approach in which **testing activities are performed continuously** along with development in **short iterations**, focusing on **customer requirements, collaboration, and adaptability to change**.

3. Key Features of Agile Testing

- Testing is continuous
- Early involvement of testers
- Customer collaboration
- Fast feedback
- Flexible to changing requirements

4. Agile Testing Process

Agile testing is performed in **iterations (sprints)**.

Agile Testing Life Cycle

Requirement Analysis

↓

Test Planning

↓

Test Case Design

↓

Test Execution

↓

Defect Reporting

↓

Feedback & Improvement

This cycle repeats for **every sprint**.

5. Role of Tester in Agile

In Agile, testers are **part of the development team**.

Responsibilities:

- Participate in sprint planning
- Review user stories
- Design test cases early
- Perform continuous testing
- Provide rapid feedback

6. Agile Testing Levels

- Unit Testing
- Integration Testing
- System Testing
- Acceptance Testing

Testing occurs **simultaneously** with development.

7. Agile Testing Types

- Functional Testing
- Regression Testing
- Exploratory Testing
- Acceptance Testing
- Performance Testing

8. Agile Testing Principles

Agile Testing follows Agile values and principles:

1. Continuous testing ensures quality
2. Testing is not a separate phase
3. Whole team is responsible for quality
4. Fast feedback is essential
5. Working software is the primary measure of progress
6. Adaptability to change

9. Diagram: Agile Testing in Scrum

Product Backlog



Sprint Planning



Sprint (2–4 Weeks)



Development + Testing



Sprint Review



Release

10. Advantages of Agile Testing

- Early defect detection
- Faster time to market
- Better customer satisfaction
- Improved collaboration
- High flexibility

11. Challenges of Agile Testing

- Frequent changes in requirements
- Limited documentation
- Time constraints
- Requires skilled testers

12. Example of Agile Testing

For an **online banking application**, testing is done for:

- Login feature in Sprint 1
- Fund transfer in Sprint 2
- Bill payment in Sprint 3

Testing and development occur **simultaneously**.

13. Agile Testing vs Traditional Testing (Short)

Agile Testing

Continuous

Iterative

Customer involvement

Traditional Testing

Sequential

Phase-based

Limited involvement

Agile Testing

Flexible

Traditional Testing

Rigid

Agile Testing is a **modern, flexible, and collaborative testing approach** that ensures **high-quality software** by integrating testing throughout the development cycle. It supports **rapid development, continuous feedback, and customer satisfaction**, making it ideal for today's dynamic software projects.

Difference between Traditional and Agile testing,

1. Introduction

Software testing approaches vary based on the **software development model** used. **Traditional Testing** follows a **sequential development model** (such as Waterfall), whereas **Agile Testing** follows an **iterative and incremental model**. Understanding the differences helps in selecting the **right testing strategy** for a project.

2. Traditional Testing

Definition

Traditional Testing is a testing approach where **testing is performed after the development phase**, following a **fixed and sequential process**.

Characteristics

- Testing is a separate phase
- Detailed documentation
- Limited customer involvement
- Changes are difficult to accommodate

3. Agile Testing

Definition

Agile Testing is a testing approach where **testing is integrated with development** and performed **continuously in iterations (sprints)**.

Characteristics

- Continuous testing
- Early involvement of testers
- High customer collaboration
- Flexible to changes

4. Diagram: Traditional vs Agile Testing

Traditional Testing Model

Requirements → Design → Development → Testing → Deployment

Agile Testing Model

Plan → Develop → Test → Review → Release

(Repeated in Sprints)

5. Comparison Table: Traditional Testing vs Agile Testing

Aspect	Traditional Testing	Agile Testing
Development Model	Waterfall / V-Model	Agile / Scrum
Testing Phase	After development	Along with development
Requirement Changes	Difficult to handle	Easily adaptable
Documentation	Heavy documentation	Minimal documentation
Customer Involvement	Limited	Continuous
Feedback	Late feedback	Early and continuous
Test Planning	Done once	Done every sprint
Test Execution	Sequential	Iterative
Automation	Limited	Highly encouraged
Delivery	Single release	Frequent releases

6. Role of Tester

Traditional Testing

- Tester works after coding
- Follows predefined test cases

Agile Testing

- Tester works with developers
- Participates in sprint planning
- Performs exploratory testing

7. Advantages

Traditional Testing

- Well-defined structure
- Suitable for stable requirements

Agile Testing

- Early defect detection
- Faster time to market

- Better quality and customer satisfaction

8. Limitations

Traditional Testing

- Late defect detection
- Rigid process

Agile Testing

- Requires experienced team
- Less documentation

9. Example

Traditional Testing:

Testing a payroll system after full development.

Agile Testing:

Testing login, salary calculation, and report modules in separate sprints.

10. Conclusion

Traditional Testing is suitable for projects with **fixed and well-defined requirements**, whereas Agile Testing is ideal for projects with **frequent changes and rapid delivery needs**. Agile Testing provides **better adaptability, early feedback, and higher software quality**.

Agile principles and values,

Agile Values

The Agile Manifesto (2001) defines four core values that guide Agile development:

1. **Individuals and interactions over processes and tools**
 - Focus on people and communication rather than rigid processes or tools.
 - Strong collaboration among team members, stakeholders, and customers is prioritized.
2. **Working software over comprehensive documentation**
 - The goal is to deliver functional software frequently, not just documentation.
 - Documentation is still important but secondary to actual working solutions.
3. **Customer collaboration over contract negotiation**
 - Continuous engagement with the customer ensures the product meets their needs.
 - Flexibility is preferred over strict adherence to initial contracts.
4. **Responding to change over following a plan**
 - Agile embraces change, even late in development.

- Teams adapt to changing requirements rather than rigidly following a plan.

Key Idea: While Agile acknowledges the value of the items on the right, it gives **more importance** to the items on the left.

Agile Principles

There are **12 principles** behind the Agile Manifesto:

- 1. Customer satisfaction through early and continuous delivery**
 - Deliver valuable software frequently to satisfy customer needs.
- 2. Welcome changing requirements**
 - Even late changes are acceptable; Agile is flexible to evolving requirements.
- 3. Deliver working software frequently**
 - Short delivery cycles (iterations/sprints) of 1–4 weeks are preferred.
- 4. Business and developers must work together daily**
 - Collaboration ensures alignment and reduces misunderstandings.
- 5. Build projects around motivated individuals**
 - Trust, support, and autonomy lead to better results.
- 6. Face-to-face conversation is the most effective communication**
 - Direct communication reduces ambiguity compared to written communication.
- 7. Working software is the primary measure of progress**
 - Progress is measured by functional product, not by documentation or plans.
- 8. Sustainable development pace**
 - Teams should maintain a constant pace indefinitely, avoiding burnout.
- 9. Continuous attention to technical excellence and good design**
 - Quality and maintainable code improve agility and reduce defects.
- 10. Simplicity—the art of maximizing work not done—is essential**
 - Focus on what is essential and avoid unnecessary work.
- 11. Self-organizing teams generate the best architectures, requirements, and designs**
 - Teams that make their own decisions perform better and are more creative.
- 12. Regular reflection and adaptation**
 - Teams should periodically reflect on how to become more effective and adjust accordingly.

Diagram (Textual Representation)

Agile Values:

Agile Principles:

- | | |
|----------------------------|--------------------------|
| 1. Individuals > Processes | 1. Customer satisfaction |
| 2. Working software > Docs | 2. Welcome changes |

3. Customer collaboration
4. Responding to change
5. Motivated individuals
6. Face-to-face conversation
7. Working software measures progress
8. Sustainable pace
9. Technical excellence
10. Simplicity
11. Self-organizing teams
12. Reflect and adapt

Agile Testing Quadrants,

Agile Testing Quadrants

Agile Testing Quadrants, proposed by Brian Marick, provide a **framework for testing in Agile projects**. They help teams understand **what to test, why, and how**, balancing business and technical needs.

The quadrants are usually visualized as **two axes**:

- **Horizontal axis:** Technology-facing vs Business-facing
- **Vertical axis:** Support the team vs Critique the product

This results in **four quadrants**:

Q1 – Technology-facing, Support the Team

- **Purpose:** Ensure the code works correctly; guide development.
- **Focus:** Automated tests, unit tests, component tests.
- **Examples:**
 - Unit testing (JUnit, NUnit)
 - Component/integration testing
 - Test-Driven Development (TDD)
- **Key Question:** “Does the code work as expected?”

Q2 – Business-facing, Support the Team

- **Purpose:** Confirm that requirements are correctly implemented.
- **Focus:** Functional tests that guide development.
- **Examples:**
 - Functional tests

- Story tests / Acceptance Test-Driven Development (ATDD)
- Prototypes, simulations
- **Key Question:** “Does the product meet business expectations?”

Q3 – Business-facing, Critique the Product

- **Purpose:** Evaluate the product from the user’s perspective; provide feedback for improvement.
- **Focus:** Exploratory testing, usability, and acceptance.
- **Examples:**
 - User acceptance testing (UAT)
 - Exploratory testing
 - Alpha/Beta testing
- **Key Question:** “Does the product satisfy the customer?”

Q4 – Technology-facing, Critique the Product

- **Purpose:** Assess technical quality, performance, and security.
- **Focus:** Non-functional tests that critique the system.
- **Examples:**
 - Performance testing
 - Load and stress testing
 - Security testing
 - Scalability testing
- **Key Question:** “Is the product robust, reliable, and maintainable?”

Diagram (Text Representation)

Business-Facing

| Q2 | Q3

| Support Team | Critique Product

Technology- | Q1 | Q4

Facing | Support Team | Critique Product

- **Q1 & Q2:** Support the team (help developers build it right)
- **Q3 & Q4:** Critique the product (evaluate quality and fitness)

Key Points:

- Quadrants encourage a **balanced testing approach**: both automated & manual, technical & business-focused.
- Helps teams decide **what tests to automate**, what tests to run manually, and **how to involve customers** in testing.

Automated Tests.

Automated Tests

Automated testing is the process of using software tools to **execute test cases automatically**, compare actual results with expected results, and report outcomes. It reduces manual effort, increases accuracy, and allows frequent testing.

Advantages of Automated Testing

1. **Faster Execution:** Tests run much quicker than manual testing, especially repetitive ones.
2. **Reusability:** Test scripts can be reused across multiple releases.
3. **Consistency:** Eliminates human error and ensures consistent execution.
4. **Continuous Integration Support:** Works well with CI/CD pipelines for Agile development.
5. **Regression Testing:** Efficient for checking old functionality after changes.
6. **Cost-effective in Long Run:** Though initial setup is expensive, it saves time and effort in the long term.

Disadvantages of Automated Testing

1. **High Initial Cost:** Setting up automation tools and scripts can be expensive.
2. **Maintenance Overhead:** Test scripts must be updated with changes in the application.
3. **Not Suitable for UI Changes:** Frequent UI updates can break automated scripts.
4. **Requires Skilled Resources:** Test automation needs programming knowledge.

Types of Automated Tests

1. **Unit Testing**
 - Tests individual components or functions of the software.
 - Example tools: JUnit, NUnit, TestNG
2. **Integration Testing**
 - Tests interaction between different modules or services.
 - Example tools: JUnit, Postman (API testing), SoapUI
3. **Functional/UI Testing**
 - Validates software behavior against requirements.
 - Example tools: Selenium, Cypress, QTP/UFT
4. **Regression Testing**
 - Ensures that new changes do not break existing functionality.
 - Example tools: Selenium, TestComplete
5. **Performance Testing**
 - Measures speed, scalability, and reliability.
 - Example tools: JMeter, LoadRunner
6. **Security Testing**
 - Detects vulnerabilities and ensures protection against threats.
 - Example tools: OWASP ZAP, Burp Suite

Automated Testing Tools (Popular)

Tool	Type	Features
Selenium	Functional/UI testing	Open-source, supports multiple browsers & languages
JUnit	Unit testing	Java-based, integrates with CI tools
TestNG	Unit & Integration	Parallel execution, flexible reporting
QTP/UFT	Functional/UI testing	Supports keyword-driven testing, enterprise-friendly
JMeter	Performance testing	Load, stress, and performance measurement
Postman	API testing	Automation of REST API tests

Best Practices

1. Automate **repetitive and high-risk tests** first.
2. Keep tests **independent and maintainable**.
3. Integrate automated tests with **CI/CD pipelines**.
4. Review and **update scripts regularly** to match application changes.
5. Use **data-driven testing** for comprehensive coverage.

1. Agile Values (Agile Manifesto)

The **Agile Manifesto** defines **4 core values** that guide Agile software development.

Agile Values

1. **Individuals and interactions** over processes and tools
→ Emphasizes communication and teamwork.
2. **Working software** over comprehensive documentation
→ Deliver functional software frequently.
3. **Customer collaboration** over contract negotiation
→ Continuous customer involvement.
4. **Responding to change** over following a plan
→ Adaptability to changing requirements.

Note: Items on the right still have value, but items on the left are valued more.

2. Agile Principles

Agile is supported by **12 principles**:

1. Customer satisfaction through early and continuous delivery
2. Welcome changing requirements, even late in development
3. Deliver working software frequently
4. Business people and developers work together daily
5. Build projects around motivated individuals
6. Face-to-face communication is most effective
7. Working software is the primary measure of progress
8. Sustainable development pace
9. Continuous attention to technical excellence
10. Simplicity – maximize work not done
11. Self-organizing teams produce best designs
12. Regular reflection and improvement

3. Agile Testing

Definition

Agile Testing is a **continuous testing approach** where testing is performed **throughout the Agile lifecycle**, not as a separate phase.

Key Characteristics

- Early testing

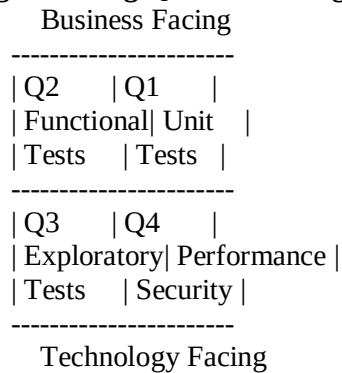
- Continuous feedback
- Collaboration with developers and customers
- Automation preferred
- Incremental and iterative testing

4. Agile Testing Quadrants

The **Agile Testing Quadrants** categorize tests based on:

- **Business vs Technology focus**
- **Support team vs Critique product**

Agile Testing Quadrants Diagram (Textual Representation)



Quadrant Description

◆ Quadrant Q1 – Technology Facing, Support Team

- Unit tests
- Component tests
- Written by developers
- Mostly automated

Examples:

- JUnit tests
- PyTest unit tests

◆ Quadrant Q2 – Business Facing, Support Team

- Functional tests
- Acceptance tests
- User story validation
- Can be manual or automated

Examples:

- Selenium functional tests
- BDD tests (Cucumber)

◆ Quadrant Q3 – Business Facing, Critique Product

- Exploratory testing
- Usability testing
- User acceptance testing (UAT)
- Mostly manual

Examples:

- Ad-hoc testing
- Scenario-based testing

◆ Quadrant Q4 – Technology Facing, Critique Product

- Performance testing
- Load testing
- Security testing
- Reliability testing

Examples:

- JMeter performance tests
- Security scans

5. Automated Testing in Agile

Definition

Automated testing uses tools and scripts to **automatically execute test cases**, compare results, and report defects.

Why Automation is Important in Agile

- Frequent releases
- Continuous integration
- Faster feedback
- Reduced regression testing time
- Improved test coverage

Types of Automated Tests

Test Type	Description	Tools
Unit Tests	Test individual functions	JUnit, PyTest
Integration Tests	Test module interactions	PyTest
Functional Tests	Test system functionality	Selenium
Regression Tests	Ensure no new defects	Selenium
Performance Tests	Check speed and load	JMeter

Automation Testing Pyramid (Agile)

UI Tests (Few)

Service/API Tests

Unit Tests (Many)

- More **unit tests**
- Fewer **UI tests**
- Faster and cost-effective

6. Manual vs Automated Testing (Agile)

Manual Testing	Automated Testing
Suitable for exploratory testing	Suitable for regression
Human observation	Tool-driven
Time-consuming	Fast execution
Not reusable	Reusable scripts